

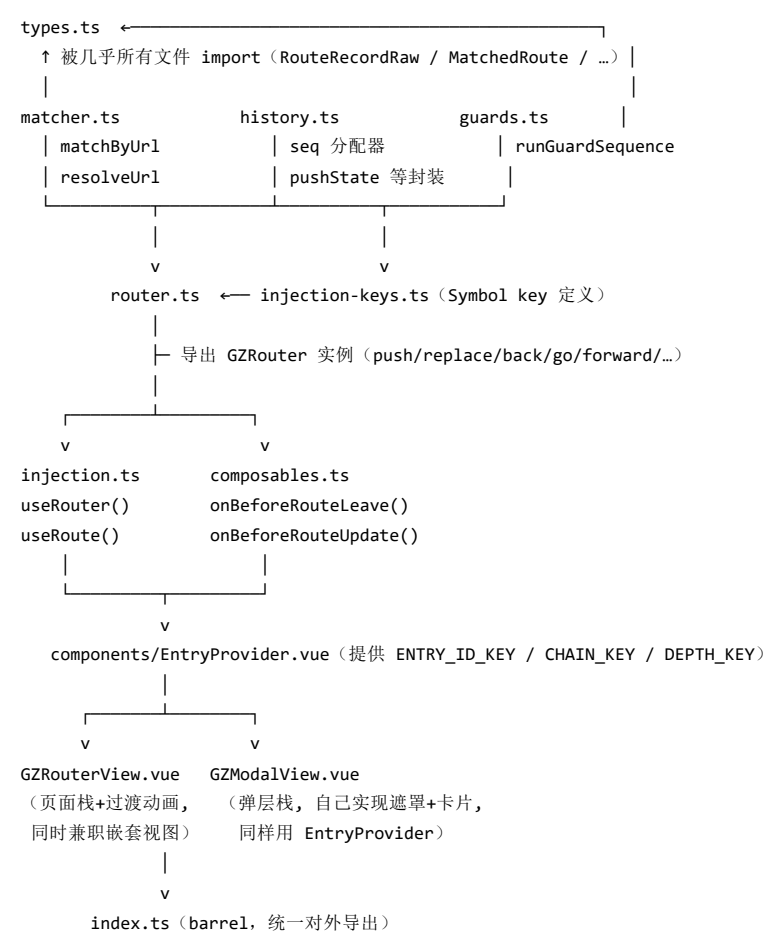
ARCHITECTURE.md — gz-vue-router

这份文档回答两个问题：每个文件具体负责什么、互相怎么调用，以及一次典型操作（启动、导航、物理返回、渲染嵌套路由……）代码是按什么顺序跑的。

- 想知道"这个包能做什么、怎么用"，看 [README.md](#)。
- 想知道"某个设计为什么长这样、踩过什么坑"，看 [CLAUDE.md](#)。
- 这份文档是"横切面"视角：按文件分工 + 按时间顺序，把 README 和 [CLAUDE.md](#) 里提到的机制串成一条能跟着读代码的主线。

维护须知同 [CLAUDE.md](#)：新增文件、或者改变了模块之间的调用关系/执行顺序，需要同步更新这份文档对应的图和小节；单纯的 bug 修复不需要动这里（除非修复本身改变了执行顺序，比如 0004/0005 那种）。

一、文件依赖关系



依赖方向始终是"下层不知道上层的存在"：types.ts 不 import 任何其它模块；matcher.ts / history.ts / guards.ts 只依赖 types.ts；router.ts 依赖前三者但不知道有 Vue 组件这回事；组件层（EntryProvider / GZRouterView / GZModalView）通过 injection.ts 拿到 router.ts 造出来的实例，反过来 router.ts 完全不 import 任何 .vue 文件。

二、文件清单与职责

文件	职责	
types.ts	所有公开/内部类型定义。不含任何逻辑。	RouteRecordRaw、MatchedRoute、Stac

文件	职责	
matcher.ts	路由匹配：把 RouteRecordRaw[] （含嵌套 children ） 展开成扁平的"完整路径模板 + 根到叶记录链"列表，并基于此做 URL 匹配和命名路由 -> URL 的反向编译。	matchByUrl1、resolveUrl1、findRouteB
history.ts	只管两件事：seq 计数器的分配与持久化、浏览器 history API 的薄封装。 不知道页面栈/弹层栈的存在。	createSeqAllocator、pushHistory / req
guards.ts	守卫执行引擎：给一组 NavigationGuard，依次跑、支持两种调用签名、 遇到取消/重定向就短路返回。不知道 push/replace/popstate 的存在。	runGuardSequence
router.ts	核心：把上面三个模块的能力组装成一个 GZRouter 实例——响应式状态 （ stack / modalStack / currentRoute / direction / canGoBack ）、导航方法 （ push / replace / back / go / forward ）、守卫注册 （ beforeEach / afterEach /内部的 leave/update 守卫登记表）、浏览器同步 （ popstate 监听）、Vue 插件安装（ install ）。	createGZRouter、GZRouter 类型
injection-keys.ts	所有 provide / inject 用的 Symbol key 集中定义， 避免各处各写各的字符串 key。	ROUTER_KEY、ROUTE_KEY、ENTRY_ID_KE
injection.ts	面向组件的取值入口： useRouter() / useRoute()，本质是对 inject(ROUTER_KEY) / inject(ROUTE_KEY) 的包装 + 缺失时的报错提示。	useRouter、useRoute
composables.ts	面向组件的守卫注册入口： onBeforeRouteLeave() / onBeforeRouteUpdate()， 找到当前组件所属的 entryId （ inject(ENTRY_ID_KEY) ），登记到 router.ts 内部的 leaveGuardsMap / updateGuardsMap， 组件卸载时自动注销。	onBeforeRouteLeave、onBeforeRouteUpd
components/EntryProvider.vue	每渲染一个 StackEntry （页面或弹层）就套一层： provide 这个 entry 的 id、路由记录链 chain、深度 1。 是连接"响应式状态里的一条数据"和"具体某一坨 DOM/组件树"的胶水层。	默认导出（内部组件，不在 index.ts 且
components/GZRouterView.vue	双重身份： 根视图 ——用在 App.vue 里，读 router.stack，<TransitionGroup> 做前进/后退滑动过渡； 嵌套视图 —— 用在某个布局组件自己的模板里，靠 inject 到的 DEPTH_KEY 判断自己是嵌套的，直接渲染路由记录链里对应深度的组件， 不做任何动画。	默认导出，index.ts 里重导出为 GZRouter
components/GZModalView.vue	渲染 router.modalStack：自己实现半透明遮罩 + 卡片容器（不依赖 Vuetify <v-dialog>），<TransitionGroup> 处理多层弹层的开关动画， 点遮罩关闭统一调用 router.back()。	默认导出，index.ts 里重导出为 GZMod
index.ts	包的公开 API 清单（barrel export）。业务代码只应该从这一个文件 import，不直接 import 内部路径。	见下方"公开 API 一览"
styles/transitions.css	<GZRouterView> 页面过渡动画用到的 CSS class （ gz-page-fwd-* / gz-page-bwd-* ）和对应的 @keyframes。纯样式，无 JS。	（CSS，无导出）

公开 API 一览（ index.ts 导出的东西）

```
createGZRouter(options): GZRouter // 工厂函数
useRouter(): GZRouter // 组件里取路由实例
useRoute(): RouteLocationNormalized // 组件里取响应式当前路由
onBeforeRouteLeave(guard) // 组件级离开守卫
onBeforeRouteUpdate(guard) // 组件级更新新守卫
resolveUrl(routes, to) // 命名路由 -> URL 字符串（一般用不上，router.push 内部会调）
GZRouterView // 页面栈渲染组件（根用法 + 嵌套用法）
GZModalView // 弹层栈渲染组件
// 以下都是纯类型
RouteRecordRaw, RouteLocationRaw, RouteLocationNormalized, RouteMeta,
NavigationGuard, NavigationGuardReturn, AfterHook, Direction, GZRouterOptions, StackEntry
```

三、核心数据结构

```
// 路由配置（用户写的路由表）
interface RouteRecordRaw {
  path: string; name: string; component: Component;
  meta?: RouteMeta; // modal / modalStyle / title / 自定义字段
  beforeEnter?: NavigationGuard | NavigationGuard[];
  children?: RouteRecordRaw[]; // 嵌套路由
}
```

```
// 一次匹配的结果（matcher.ts 产出，router.ts 消费）
interface MatchedRoute {
  route: RouteRecordRaw; // 叶子记录，chain 最后一个元素的别名，方便单独取用
  chain: RouteRecordRaw[]; // 根到叶完整记录链，未嵌套时 = [route]
  params: Record<string, string>;
  path: string; url: string;
}
```

```
// 页面栈/弹层栈里的一条历史记录
interface StackEntry {
  id: number; // 全局递增，Vue :key 用它，onBeforeRouteLeave/Update 也按它登记
  seq: number; // 浏览器历史序号，页面栈和弹层栈共用同一个分配器
  matched: MatchedRoute;
}
```

GZRouter （createGZRouter() 返回值， router.ts 内部维护的响应式状态）：

```
{
  currentRoute, // reactive 对象，字段随导航原地更新
  stack, // shallowReactive<StackEntry[]>, 页面栈
  modalStack, // shallowReactive<StackEntry[]>, 弹层栈
  direction, // ref<'forward'|'backward'|'none'>, 只服务于页面栈过渡动画
  canGoBack, // computed<boolean>
  push, replace, back, forward, go,
  beforeEach, afterEach,
  install, // Vue 插件安装函数
  _registerLeaveGuard / _unregisterLeaveGuard, // @internal, composables.ts 用
  _registerUpdateGuard / _unregisterUpdateGuard, // @internal, composables.ts 用
}
```

四、执行顺序：应用启动

main.ts

```
createGZRouter({ routes, initialUrl, syncBrowserHistory }) // router.ts
├─ new seq 分配器 (history.ts createSeqAllocator)
├─ 读 window.location (如果 syncBrowserHistory) 算出 startUrl
├─ 读 window.history.state 看有没有已存在的 seq (刷新场景会有)
├─ initLanding()
│   └─ matchByUrl(routes, startUrl) 匹配当前 URL // matcher.ts
│   └─ 命中弹层路由 -> 背景页塞进 stack, 弹层塞进 modalStack
│   └─ 命中普通路由 -> 直接塞进 stack
│   └─ Object.assign(currentRoute, toLocation(matched)) // meta 按 chain 合并
│       └─ queueMicrotask(() => 跑 afterEachHooks) // 见下方"为什么延后"
├─ replaceHistory(initialSeq, startUrl) // history.ts
└─ 注册 window.addEventListener('popstate', ...) (如果 syncBrowserHistory)
```

返回 router 实例

应用代码: router.beforeEach(...) / router.afterEach(...) // 同步注册, 晚于上面的 initLanding

(这也是为什么 initLanding 里用 queueMicrotask 延后触发 afterEach—

要等这些注册代码跑完, 微任务才轮到执行)

createApp(App).use(router).mount('#app')

app.use(router) -> 调用 router.install(app)

├─ app.provide(ROUTER_KEY, router)

└─ app.provide(ROUTE_KEY, currentRoute)

(故意不设置 app.config.globalProperties.\$router, 原因见 CLAUDE.md 设计决策 10)

挂载阶段:

App.vue 渲染 <GZRouterView/> + <GZModalView/>

GZRouterView.vue setup()

inject(DEPTH_KEY) -> undefined -> 判定自己是"根视图"

useRouter() -> ROUTER_KEY 拿到实例

visibleEntries = computed(stack.slice(-2)) -> 此时 stack 已有 initLanding 塞的那一条

渲染 EntryProvider(entryId, chain, params) 包一层 -> chain[0].component

GZModalView.vue setup() 同理, 读 modalStack (可能是空的)

五、执行顺序： router.push(to)

push(to)

```
url = resolveUrl(routes, to)          // matcher.ts: 字符串直接用: {name,params,query} 编译成 URL
matched = matchByUrl(routes, url)     // matcher.ts: 拿到 { route, chain, params, path, url }
matched 是弹层路由? -> 转给 pushModal(matched, url), 流程见下节, 到此为止
outgoing = modalStack 为空 时的 stack 当前 top (否则 null—有弹层盖着, 背景页不算"离开")
guards = [
  outgoing 的 leave 守卫 (如果有),
  全局 beforeEach,
  chain 上所有记录的 beforeEnter (祖先先跑),      // collectChainGuards
]
withGuards(guards, toLocation(matched), mutate):
  from = currentRoute 快照
  outcome = runGuardSequence(guards, to, from)    // guards.ts, 依次跑, 某个返回 false/重定向就短路
  不放行:
    有重定向目标 且 目标 !== currentRoute.fullPath -> 递归调用 push(redirectUrl)
    返回 false, 什么都不 mutate
  放行:
    mutate()    // 调用方传进来的闭包, 实际执行:
      seq = seqAllocator.next()
      direction.value = 'forward'
      stack.push(makeEntry(matched, seq))          // shallowReactive 数组 push, 触发响应式
      currentSeq.value = seq
      pushHistory(seq, url)                        // history.ts: window.history.pushState
      Object.assign(currentRoute, to)              // 更新响应式当前路由
      跑所有 afterEachHooks(to, from)
      返回 true
```

Vue 响应式生效:

GZRouterView 的 visibleEntries 重新计算 -> 多了一条 entry

TransitionGroup 检测到新 key -> 新 entry 的 DOM 走 gz-page-fwd-enter-active

旧 entry (现在退居第二位) 走 CSS class 切换 (is-current -> is-previous)

六、执行顺序： router.replace(to) （含嵌套路由的"布局保留"分支）

```
replace(to)
  url, matched 同 push
matched 是弹层? -> 转给 replaceModal, 流程和下面对称 (比较 topModal 而不是 top)
currentTop = stack 当前 top
isUpdate = currentTop 存在
  且 modalStack 为空
  且 currentTop.matched.chain[0].component === matched.chain[0].component
↑ 比较链根 (chain[0]), 不是叶子 (route) —
  未嵌套路由 chain[0]===route, 这个条件退化成"和之前完全一样的叶子比较";
  嵌套路由场景下, 只要共享同一个布局根, 就算 isUpdate

guards = [
  isUpdate ? currentTop 登记的 update 守卫 (onBeforeRouteUpdate 注册的)
    : currentTop 登记的 leave 守卫 (onBeforeRouteLeave 注册的),
  全局 beforeEach,
  collectChainGuards(matched),
]
withGuards(..., mutate):
  mutate() 内部:
    seq = next()
    newEntry = makeEntry(matched, seq, isUpdate ? currentTop.id : undefined)
      ↑ isUpdate 为真时复用同一个 id—Vue :key 不变,
      EntryProvider/chain[0] 组件实例不重建
    stack.splice(indexOf(currentTop), 1, newEntry) // 原地替换这一条 entry
    replaceHistory(seq, url) // 不产生新的浏览器历史记录
Vue 响应式生效 (isUpdate 为真、嵌套路由场景):
  entry.id 没变 -> GZRouterView 根视图那层 <component :is="entry.matched.chain[0].component">
    的 vnode key 没变 -> 布局组件实例不重建, 只是 patch 了 v-bind 的 params
  EntryProvider 的 CHAIN_KEY 是 ComputedRef -> chainCtx.value 变成了新的 chain/params
  布局组件模板里嵌套的 <GZRouterView> (inject 到了 DEPTH_KEY):
    nestedComponent = computed(() => chainCtx.value.chain[depth]?.component)
  这个 computed 重新求值 -> 如果新旧 chain[depth] 是不同组件 -> <component :is> 类型变了
    -> Vue 卸载旧的子路由组件、挂载新的
    -> 布局本身 (chain[0]) 完全没有被触碰
```

七、执行顺序： router.back()

back() 内部是四个互斥分支, 按顺序判断:

```
back(fallbackUrl = initialUrl)
```

③ modalStack.length > 0

```
leaving = 最上层弹层
below   = 弹层栈里它下面那一条（可能没有，那就是背景页 top.value）
guards  = leaving 的 leave 守卫 + beforeEach
withGuards(..., to=toLocation(below 或 top), mutate):
    modalStack.pop()
    currentSeq 更新
    canGoBack ? goBackInHistory()           // 真的有历史可退
                : replaceHistory(seq, below/top 的 url) // 没历史，地址栏静默落回原地
→ return（不会继续往下判断）
```

④ stack.length > 1（没有弹层挡着，页面栈本身有上一页）

```
leaving = 当前 top
guards  = leaving 的 leave 守卫 + beforeEach
withGuards(..., to=toLocation(stack 里倒数第二条), mutate):
    direction.value = 'backward'
    stack.pop()
    goBackInHistory()
→ return
```

⑤ canGoBack.value 为真（内存里没有上一页，但浏览器 session history 里确实有一多半是刷新丢的）

凑不出精确的 to（浏览器不暴露上一条历史对应的路由），先用 UNKNOWN_LOCATION 占位跑 leave 守卫 + beforeEach 放行的话只是 goBackInHistory()—真正的目标路由靠接下来触发的 popstate 事件去解析（见下节）

```
→ return
```

⑥ 以上都不成立：这个 tab 里压根没有更早的历史

```
matchByUrl(routes, fallbackUrl) 匹配兜底路由（默认就是 initialUrl）
guards = leaving 的 leave 守卫 + beforeEach（这次能算出精确的 to，所以不需要在 popstate 里补）
withGuards(..., mutate):
    stack 整体替换成只有这一条 fallback entry（不是 push，是 splice(0, length, ...)）
    replaceHistory(seq, fallbackUrl)
```

八、执行顺序：物理前进/后退（popstate 事件）

这是最复杂的一段，因为浏览器只告诉你“跳到了 state.seq 是多少的那条记录”，剩下的全靠自己拼。分三层判断：

```

window.addEventListener('popstate', (event) => {
  state = event.state  // { seq: number } | null

  ④ modalStack 里能找到 state.seq (在 modalStack[modalIndex])
  且 modalIndex < modalStack.length - 1 (说明是往栈里更靠前的位置退，必然是"后退")：
    guards = 当前最上层弹层的 leave 守卫 + beforeEach
    withGuards(..., to=modalStack[modalIndex] 对应的路由, mutate):
      modalStack.splice(modalIndex + 1)  // 一次性截断到目标层级，不管一次跳了几级
      不放行 -> goInHistory(1)  // 撤销这次后退 (方向永远是 +1，因为这个分支必然是后退)
    → return

  ⑤ stack 里能找到 state.seq (在 stack[pageIndex])
  且 (pageIndex 不是当前 top，或者当前挂着弹层)：
    guards = (只有没挂弹层时才算) 当前 top 的 leave 守卫 + beforeEach
    withGuards(..., to=stack[pageIndex] 对应的路由, mutate):
      direction.value = 'backward'
      modalStack 整体清空  // 任何弹层的 seq 必然比这条页面记录大，回到它就等于关掉所有弹层
      stack.splice(pageIndex + 1)
      不放行 -> goInHistory(1)
    → return

  ⑥ 两个栈都找不到 state.seq：多半是刷新/直接深链接丢了内存历史
    matched = matchByUrl(routes, 当前 location)  // 唯一可信的信息来源
    goingBackward = state.seq < (当前 modalStack 顶 或 stack 顶 的 seq)
    matched 是弹层路由：
      guards = beforeEach + collectChainGuards(matched)
      withGuards(..., mutate): modalStack.push(新 entry)  // 叠加，不整体替换 (0005 修的就是这里)
      不放行 -> goInHistory(goingBackward ? 1 : -1)
    matched 是普通路由：
      guards = 当前 top 的 leave 守卫 + beforeEach + collectChainGuards(matched)
      withGuards(..., mutate):
        direction.value = goingBackward ? 'backward' : 'forward'
        modalStack 清空
        stack 整体替换成只有这一条新 entry
        不放行 -> goInHistory(goingBackward ? 1 : -1)
  })

```

三层判断的优先级本质上是"先看弹层栈、再看页面栈、最后兜底重新匹配"——因为 seq 是页面栈和弹层栈共用的一条时间线，任何一个 state.seq 要么落在其中一个栈里、要么两个都找不到，不会有第三种情况。

九、执行顺序：onBeforeRouteLeave / onBeforeRouteUpdate 的注册与触发

组件 setup() 执行期间：

```

onBeforeRouteLeave(guard)  // composables.ts
getCurrentInstance() 拿到组件实例 (校验必须在 setup 里调)
inject(ROUTER_KEY) 拿路由实例
inject(ENTRY_ID_KEY, null) 拿这个组件所属的 entry id
↑ 这个值是外层 EntryProvider 在渲染这个 entry 时 provide 的，
  不管这个组件是 chain[0] (布局) 还是 chain[depth] (嵌套的子路由)，
  inject 到的都是同一个 entryId——因为 provide/inject 沿组件树往下传，
  中间隔几层嵌套的 <GZRouterView> 不会重新 provide 一个新的 ENTRY_ID_KEY
router._registerLeaveGuard(entryId, guard)  // 塞进 router.ts 内部的 leaveGuardsMap
onUnmounted(() => router._unregisterLeaveGuard(...))  // 组件卸载时自动清理

```

导航发生时 (push/replace/back/popstate 任一分支)：

```

guards 数组拼接时会读 leaveGuardsMap.get(entryId) 或 updateGuardsMap.get(entryId)
→ 这些守卫和 beforeEach/beforeEnter 一起，被 runGuardSequence 依次调用
→ 任何一个返回 false，整个导航 (不只是这个组件) 都会被取消

```


十、执行顺序：嵌套路由渲染（<GZRouterView> 的双重身份）

以 /dashboard/stats （DashboardLayout 是 chain[0]，DashboardStats 是 chain[1]）为例：

```
根 <GZRouterView>（App.vue 里的那个）
inject(DEPTH_KEY) -> undefined -> isNested = false，走"根视图"渲染路径
visibleEntries 包含这条 entry（matched.chain = [DashboardLayout，DashboardStats]）
渲染：
  <EntryProvider :entry-id :chain :params>      // provide ENTRY_ID_KEY / CHAIN_KEY(ComputedRef) / DEPTH_KEY=1
    <component :is="chain[0].component" />      // 即 DashboardLayout，v-bind 整个 params
  </EntryProvider>
```

```
DashboardLayout 组件挂载，模板里写了一个 <GZRouterView />（嵌套用法）
这第二个 <GZRouterView> 实例 setup() 执行：
  inject(DEPTH_KEY) -> 1（EntryProvider 刚 provide 的）-> isNested = true
  inject(CHAIN_KEY) -> ComputedRef<{chain, params}>
  provide(DEPTH_KEY, 2)  // 给"万一 DashboardStats 自己里面还嵌套了一层"的场景铺路
  nestedComponent = computed(() => chainCtx.value.chain[1]?.component)  // DashboardStats
  nestedParams     = computed(() => chainCtx.value.params)
渲染：
  <component :is="nestedComponent" v-bind="nestedParams" />  // 即 DashboardStats
```

用户点标签页切到 /dashboard/overview（DashboardLayout.vue 调用 router.replace({name:'dashboard-overview'})）：
见第六节 replace() 流程—isUpdate 判定为真（chain[0] 都是 DashboardLayout）
→ entry.id 不变 → 外层 <component :is="chain[0].component"> 的 vnode key 不变
 → DashboardLayout 组件实例不重建（它内部的 <GZRouterView> 实例也不重建）
→ EntryProvider 的 CHAIN_KEY（ComputedRef）重新求值 → chainCtx.value.chain[1] 变成 DashboardOverview
→ 嵌套 <GZRouterView> 的 nestedComponent 这个 computed 重新求值，值变了
 → <component :is="nestedComponent"> 类型变化 → Vue 卸载 DashboardStats、挂载 DashboardOverview
全程 DashboardLayout 自己的状态（比如它 setup() 里的本地 ref）不受影响

十一、和现有两份文档的关系

文档	回答的问题	什么时候去看
README.md	这个 API 怎么用？参数是什么？	写业务代码、集成到新项目时
CLAUDE.md	为什么这么设计？改哪里容易踩坑？怎么验证改动？	要改这个包的实现、或者怀疑一个行为是不是 bug 时
ARCHITECTURE.md (这份)	代码从哪个文件的哪个函数开始、按什么顺序执行到哪？	第一次读这个包的源码、或者要新增一个跨文件的功能（比如这次的嵌套路由）时，先建立整体地图

三份文档任何一份涉及“公开行为”的部分发生变化，都要同步检查另外两份是否也需要更新——具体规则见 [CLAUDE.md](#) 开头的维护约定。